



Universidad Don Bosco

Facultad de Ingeniería

Escuela de Computación

Guía de laboratorio Nº 7

Ciclo I

Nombre de la práctica: Expresiones regulares con JavaScript.

Lugar de ejecución: Centro de cómputo.

Materia: Desarrollo de Aplicaciones Web con Software Interpretado en el Cliente.

I Objetivos.

- ✓ Comprender la utilidad de las expresiones regulares para resolver problemas dentro de los scripts.
- ✓ Aplicar expresiones regulares para validar información ingresada en campos de formularios.
- ✓ Utilizar expresiones regulares para reemplazar texto.
- ✓ Utilizar los métodos relacionados con expresiones regulares del objeto *RegExp*.

II Introducción Teórica.

Expresiones regulares.

Las expresiones regulares son patrones usados para gestionar la concordancia o coincidencia de patrones de caracteres con cadenas de prueba. Dicho de otra forma, una expresión regular permite especificar patrones de caracteres que serán comparados con cadenas de texto con el objeto de validar datos de entrada, realizar búsquedas y/o reemplazos.

Entre las utilidades más comunes de las expresiones regulares están la comprobación de datos introducidos en formularios, la creación y el análisis de cookies, la construcción y modificación de cadenas de texto, etc.

Las expresiones regulares permiten el reconocimiento y extracción de casi cualquier tipo de datos de cadena con formato predecible como pueden ser direcciones URL, direcciones de correo electrónico, números de teléfono, números de identificación, cookies, etc.

Creación de expresiones regulares en JavaScript.

Existen dos formas de crear expresiones regulares en JavaScript, que son:

- ♦ Usando sintaxis declarativa con literales de expresiones regulares, de la siguiente forma:

```
e = /ab + c/;
```

- ♦ Llamando al constructor del objeto RegExp

```
re = new RegExp("ab + c");
```

En la primera forma los caracteres que forman el patrón van colocados entre barras de división ("/" y "/"). Por ejemplo, para crear una expresión regular que coincida con una cadena que contenga http://, deberíamos escribir:

```
var pattern = /http:\\\\//;
```

En donde se han utilizado códigos de escape para las barras de división.

Modificadores de las expresiones regulares

Los modificadores se utilizan para alterar la interpretación del patrón. Estos modificadores se colocan inmediatamente después de la barra de cierre de la expresión regular ("/"). La tabla siguiente muestra los modificadores de las expresiones regulares

CARÁCTER	Significado
i	Sin distinción entre mayúsculas y minúsculas
g	Correspondencia global, que significa que encuentra todas las correspondencias de las cadenas y no solo la primera
m	Correspondencia de líneas múltiples

Cuando se utiliza el **constructor RegExp** deben enviarse como argumentos del constructor, el patrón deseado y el modificador especial para dicho patrón introducido como primer argumento. Este segundo argumento es opcional. Veamos el siguiente ejemplo:

```
var pattern = new RegExp("http://");
```

Usando modificadores:

```
var pattern = new RegExp(http://, "i");
```

La sintaxis de constructor se utiliza con mayor frecuencia cuando el patrón a comparar no se determina hasta el momento de la ejecución. Podríamos permitir al usuario introducir la expresión regular y luego pasarla al constructor `RegExp()` como argumento.

Caracteres especiales para la creación de patrones.

Existen una serie de caracteres especiales utilizados en las expresiones regulares que sirven para definir patrones. Estos se pueden agrupar en varias categorías, entre las que se pueden mencionar:

◆ Indicadores de posición

Son caracteres especiales que dan información sobre la posición en la que tiene que estar el patrón en la cadena con la que se comparará. Los caracteres indicadores de posición utilizados en expresiones regulares de JavaScript se muestran a continuación:

CARÁCTER	Significado
^	El patrón debe aparecer al comienzo de la cadena
\$	El patrón debe aparecer al final de la cadena

Por ejemplo, el resultado de la evaluación de la expresión regular que se muestra en el siguiente segmento de código dará un resultado de falso:

```
var re = /^http/;
re.test("El protocolo es http");
```

Pero si creamos la expresión regular con el símbolo \$ el resultado será verdadero si escribimos:

```
var re = /http$/;
re.test("El protocolo es http");
```

➤ Código de escape.

Las expresiones regulares de JavaScript utilizan códigos de escape para indicar caracteres que podrían generar ambigüedad al utilizarlos dentro de la definición del patrón de una expresión regular. Cada carácter de código de escape se especifica utilizando un carácter de barra invertida (`\`). Los códigos de escape utilizados en JavaScript son un superconjunto de los utilizados en las cadenas. El conjunto completo de caracteres de códigos de escape se muestra en la siguiente tabla:

CÓDIGO	CORRESPONDENCIA CON
<code>\f</code>	Avance de página
<code>\n</code>	Nueva línea
<code>\r</code>	Retorno de carro
<code>\t</code>	Tabulador
<code>\v</code>	Tabulador vertical
<code>\/</code>	Barra inclinada
<code>\\</code>	Barra invertida
<code>\.</code>	Punto
<code>*</code>	Asterisco
<code>\+</code>	Signo más
<code>\?</code>	Signo de interrogación
<code>\ </code>	Barra horizontal
<code>\(</code>	Paréntesis de apertura
<code>\)</code>	Paréntesis de cierre
<code>\[</code>	Corchete de apertura

<code>\]</code>	Corchete de cierre
<code>\{</code>	Llave de apertura
<code>\}</code>	Llave de cierre
<code>\000</code>	Carácter ASCII representado por el valor octal
<code>\xHH</code>	Carácter ASCII representado por el valor hexadecimal HH
<code>\xHHHH</code>	Carácter Unicode representado por el valor hexadecimal HHHH

Por ejemplo, el resultado de la evaluación de la siguiente expresión regular dará como resultado falso:

```
var re = /http:\\\\//;
re.test("www.udb.edu.sv");
```

Por el contrario, la evaluación de la siguiente expresión regular dará verdadero:

```
var re = /http:\\\\//;
re.test("http://www.udb.edu.sv");
```

➤ Cuantificadores de repetición.

Estos caracteres permiten indicar la cantidad de veces que un elemento anterior al cuantificador de repetición en la expresión, puede o debe aparecer. Entiéndase por elemento el carácter o conjunto de caracteres anterior. Los cuantificadores de repetición se muestran en la siguiente tabla:

CARÁCTER	Significado
<code>*</code>	Busca coincidencia con el elemento (carácter) anterior cero o más veces.
<code>+</code>	Busca coincidencia con el elemento (carácter) anterior una o más veces.
<code>?</code>	Busca coincidencia con el elemento (carácter) anterior cero o una vez, pero no más.
<code>{m}</code>	Busca coincidencia con el elemento (carácter) anterior exactamente <i>m</i> veces.
<code>{m, n}</code>	Busca coincidencia con el elemento (carácter) anterior un mínimo de <i>m</i> veces y un máximo de <i>n</i> veces.
<code>{m, }</code>	Busca coincidencia con el elemento (carácter) anterior <i>m</i> o más veces.

Observe los siguientes ejemplos:

```
var re = /ab*c/;
```

Se obtendrá verdadero si se evalúa con las siguientes expresiones:

```
re.test("ac");
re.test("abbc");
re.test("El abc del amor");
```

◆ Agrupamiento de caracteres

JavaScript permite **agrupar caracteres** utilizando **paréntesis**. Muchas veces se requiere que el elemento anterior no sea un carácter específico sino un grupo de posibilidades de caracteres. Cualquier grupo de secuencias de códigos de expresiones regulares encerrado entre paréntesis es considerado una unidad para los operadores de las expresiones regulares. Por ejemplo:

```
var patron = /a(bc)+/;
```

Buscará correspondencias con cadenas que contengan una letra a, no necesariamente al comienzo, seguido de una o más repeticiones de "bc". En el caso de querer especificar en un patrón **clases de caracteres** como caracteres válidos se pueden mostrar un conjunto de caracteres encerrados entre corchetes. Esto indicará que se acepta como coincidente cualquier cadena que contenga alguno de esos caracteres. Por ejemplo:

```
var patron = /[1234567890]/;
```

Encontrará coincidencias con cadenas que contengan un número. Como "2227-3735", "Tenía 1", "1 gol", etc. No encontrará coincidencia con: "Primero uno", "dos tres cuatro", etc.

Al querer establecer un conjunto consecutivo de caracteres demasiado grande para mostrarlos uno a uno. Se puede utilizar el guión ("-") para indicar un rango de caracteres. Por ejemplo:

```
var patron = /[0-9]/; //Busca correspondencia con cualquier cadena que contenga número
var patron = /[a-zA-Z]/; //Busca correspondencia con cualquier carácter alfabético
var patron = /[a-zA-Z0-9]/; //Busca correspondencia con cualquier carácter alfanumérico
```

Los corchetes también se pueden utilizar para describir clases de caracteres negativos. Estas clases especifican caracteres que no pueden estar presentes. Una clase negativa se especifica anteponiendo el carácter *circunflejo* ("^") al inicio de la clase. Por ejemplo:

```
var patron = /^[^a-zA-Z]+/; //Coincidirá con secuencias de uno o más caracteres no
                             //alfabéticos
```

NOTA IMPORTANTE: Recuerde que el circunflejo también se utiliza como indicador de posición. Usted debe distinguirlo del uso como carácter negativo por el hecho de que en este último caso debe ir dentro de los símbolos de corchetes.

Existen otras clases de caracteres, como el punto (".") que se utiliza para que se acepte cualquier carácter, con excepción del salto de línea. También se puede utilizar la secuencia \s, para indicar que se acepta cualquier carácter de espacio en blanco (\t\n\r\f\v), \S para cualquier carácter que no sea espacio en blanco. \w, para cualquier carácter alfanumérico y \W, para cualquier carácter que no sea alfanumérico. \d, para cualquier carácter de dígito. \D para cualquier carácter que no sea dígito.

La siguiente tabla muestra un resumen de las clases de caracteres:

CÓDIGO	CORRESPONDENCIA CON
[caracteres]	Cualquier carácter indicado explícitamente o como un rango entre los corchetes
[^caracteres]	Cualquier carácter que no sea ninguno de los indicados entre los corchetes representados explícitamente o como rango
.	Cualquier carácter con excepción del carácter de nueva línea
\w	Cualquier carácter alfanumérico. Igual que [a-zA-Z0-9]
\W	Cualquier carácter que no sea alfanumérico. Igual que [^a-zA-Z0-9]
\s	Cualquier carácter de espacio en blanco. Lo mismo que [\t\n\r\f\v]
\S	Cualquier carácter que no sea de espacio en blanco. Lo mismo que [^ \t\n\r\f\v]
\d	Cualquier carácter de dígito. Lo mismo que [0-9]
\D	Cualquier carácter que no sea dígito. Lo mismo que [^0-9]
\b	Un límite de palabra. El espacio vacío entre un \w y \W
\B	Un límite de caracteres que no sean palabras. El espacio vacío entre caracteres de palabras
[\b]	Barra horizontal

Creación de alternativas dentro de la expresión regular

Cuando se desea crear una expresión regular en la que se quiera coincidencia con una, dos o más alternativas, tal y como ocurre con un operador lógico O, es conveniente utilizar el operador "|", que se lee como O lógico, igual que en los operadores condicionales.

Para entender el funcionamiento, imaginemos que queremos crear un patrón para que se obtenga coincidencia únicamente si la cadena comienza con *http://*, *https://* o *ftp://*. Entonces debe utilizar un patrón como el siguiente:

```
var patron = /^(http:\/\/|https:\/\/|ftp:\/\/)/;
```

Las siguientes cadenas mostrarían coincidencia:

```
http://www.google.com
https://www.ortoedu.com
http://www.ube.edu.sv
ftp://www.udb.edu.sv
```

En tanto que las siguientes cadenas no mostrarán coincidencia:

```
http://www.hotmail.com
https://www.bancodinamico.com
httpg://www.minegocio.com
mail://www.udb.edu.sv
```

Subexpresiones

En expresiones regulares una subexpresión consiste en crear una expresión regular compuesta de varias secciones individuales delimitadas por paréntesis. Cuando se tienen subexpresiones la expresión regular se compone de más de un grupo de paréntesis, de forma que cada subexpresión puede ser referida de forma individual utilizando propiedades estáticas del objeto **RegExp**.

El siguiente ejemplo ilustra el uso de las subexpresiones:

```
var customer = "Ricardo 2227-3735";  
var pattern = /(\w+) ([\d-]+)/;  
pattern.test(customer);
```

Para poder acceder de forma individual a cada una de las subexpresiones de la expresión regular anterior se hace uso de propiedades especiales utilizando la siguiente sintaxis: \$1, \$2, \$3, ..., \$9. Cada una de las propiedades anteriores recorre la expresión regular de izquierda a derecha, de modo que \$1 accede a la subexpresión de más a la izquierda (o la primera), \$2, hace referencia a la segunda subexpresión más a la izquierda (segunda), y así sucesivamente, hasta llegar a la última subexpresión.

Para el caso del ejemplo anterior podríamos escribir una sentencia como la siguiente para acceder a las dos subexpresiones y mostrar los componentes de la cadena compuesta en la variable *customer*:

```
alert("$1 = " + RegExp.$1 + "\n$2 = " + RegExp.$2);
```

NOTA IMPORTANTE: Según la especificación ECMA debería poder accederse a más de 9 subexpresiones dentro de una misma expresión regular. Sin embargo, el soporte a esta característica por parte de los navegadores no se puede asumir por defecto. Sería de probar en cada uno a ver hasta cuántas subexpresiones se puede tener acceso. La recomendación es no hacerlo a más de nueve; es decir, desde \$1 hasta \$9.

Trabajar con expresiones regulares

Cuando se trabaja con expresiones regulares en JavaScript, se utilizan los métodos **test** y **exec** del objeto **RegExp** y los métodos **match**, **replace**, **search** y **split** del objeto **String**. La utilidad de cada uno de estos métodos se muestra en la siguiente tabla:

CARÁCTER	Significado
exec	Método del objeto RegExp que realiza una búsqueda por una coincidencia en una cadena. El método exec() devuelve un arreglo (array) con información diversa. Esta información puede ser accedida mediante propiedades referidas al objeto en el que se asigna la ejecución del método exec().
test	Es el método más básico del objeto RegExp realiza un análisis en la cadena pasada como argumento para encontrar si hay coincidencia con el patrón definido. Este método devuelve un valor lógico (true o false).
match	Método del objeto String que realiza la búsqueda de una coincidencia dentro de una cadena. Retorna un arreglo con información o un valor nulo en caso de haber error.
search	Es un método del objeto String que realiza una búsqueda de un patrón dentro de una cadena. Devuelve el índice del carácter en el que empieza la primera subcadena coincidente.
replace	Este método del objeto String devuelve una cadena que resulta de reemplazar el texto que coincide con el primer argumento (una expresión regular) dentro del segundo argumento (una cadena). Si se utiliza el modificador g se reemplazan todas las coincidencias en la cadena. De lo contrario sólo se reemplaza la primera coincidencia.
split	Este método del objeto String divide una cadena en subcadenas y las devuelve en un arreglo (array).

III Materiales y Equipo.

Para la realización de la guía de práctica se requerirá lo siguiente:

No.	Requerimiento	Cantidad
1	Guía de práctica #7: Expresiones regulares con JavaScript	1
2	Computadora con Editor HTML y navegadores instalados	1

IV Procedimiento.

INDICACIONES. Cree una estructura de carpetas para almacenar en el servidor web los guiones o secuencias de comando que desarrollará durante esta guía de práctica. Se sugiere que cree una estructura con una carpeta llamada lic, dentro de la cual creará una carpeta para cada ejemplo del procedimiento, a las que puede llamar ejemplo1, ejemplo2, ..., ejemploN (donde N es un entero) y dentro de cada una de estas carpetas una para hojas de estilo con nombre css, otra para los guiones de JavaScript que puede llamar js, otra para las imágenes, si las utiliza, a la que podrá llamar img y otra para las fuentes, si las utiliza, a la que llamará fonts.

Ejemplo #1: En el siguiente ejemplo se utilizan expresiones regulares para validar distintos tipos de datos que es usual ingresar en ciertos formularios, como el DUI, NIT, tarjeta de crédito, carnet de la UDB y número de teléfono de El Salvador. Al seleccionar el tipo de dato se solicita en un campo de texto de formulario ingresar el valor correspondiente y al presionar el botón enviar se valida que el dato ingresado sea del formato correcto de acuerdo al tipo de dato seleccionado en el campo select.

Guión 1: validacion.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Validador de datos</title>
  <meta charset="utf-8" />
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.0/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-
gH2y1JqKdNHPEq0n4Mqa/HGKIhSkIHeL5AyhkYV8i59U5AR6csBvApHHNI/vl1Bx" crossorigin="anonymous">
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.2.0/dist/js/bootstrap.bundle.min.js" integrity="sha384-
A3rJD856KowSb7dwlZdYEkO39Gagi7VlsF0jrRAoQmDCKKtQBHUUlZ9AsSv4jD4Xa" crossorigin="anonymous"></script>
  <script src="js/validatedata.js"></script>
</head>
<body>
  <div class="container">
    <div class="m-4 d-flex h-100 w-100 justify-content-center align-items-center">
      <div class="col-auto bg-light p-5">
        <form action="javascript:void(0)" name="frmdatos" id="dataform">
          <h2>Validador de datos</h2>
          <div class="input-group p-2">
            <label class="form-label" for="tipo">Seleccione el tipo de dato:</label>&nbsp;&nbsp;&nbsp;
            <select class="form-select" name="seltipo" id="tipo">
              <option value="dui" selected>DUI</option>
              <option value="nit">NIT</option>
              <option value="tarjeta">Tarjeta de crédito</option>
              <option value="carnet">Carnet UDB</option>
              <option value="telefono">Teléfono</option>
            </select>
          </div>
          <div class="input-group p-2">
            <label class="form-label" for="txtdato">Dato:</label>&nbsp;&nbsp;&nbsp;
            <input class="form-control" type="text" name="txtdato" id="data" value="" maxlength="20" placeholder="(Ingrese el
dato)" /><br />
          </div>
          <input class="w-100 btn btn-primary" type="button" name="btnvalidar" id="validar" value="Validar" />
        </form>
      </div>
    </div>
  </div>
</body>
</html>
```

Crear carpeta js y dentro de ella cree y coloque el siguiente script.

Guión 2: validatedata.js

```
function iniciar(){
    var boton = document.getElementById("validar");
    if(boton.addEventListener){
        boton.addEventListener("click", validar, false);
    }
    else if(boton.attachEvent){
        boton.attachEvent("onclick", validar);
    }
}

function validar(){
    var tipo = document.frmdatos.seltipo.options[frmdatos.seltipo.selectedIndex].value;
    var dato = document.frmdatos.txtdato.value;
    var valido = false;
    var re = null;
    //alert(dato + "|" + tipo);
    if(dato == null || dato == "" || dato.length == 0){
        alert("No se ha ingresado ningún valor en el campo de formulario");
        return 0;
    }
    switch(tipo){
        //El formato de DUI en El Salvador es #####-#
        case "dui":
            re = /^\\d{8}-\\d{1}$/;
            if(re.test(dato)){
                valido = true;
            }
            break;
        //El formato del NIT en El Salvador es ####-#####-###-#
        case "nit":
            re = /^[0-9]{4}-[0-9]{6}-[0-9]{3}-[0-9]{1}$/;
            if(re.test(dato)){
                valido = true;
            }
            break;
        //Solo aplica para tarjetas de crédito Visa y Master Card en las que el formato es ####-####-####-####
        case "tarjeta":
            re = /^(\\d{4}-){3}(\\d{4}){1}$/;
            if(re.test(dato)){
                valido = true;
            }
            break;
        //El formato en los carnets de la UDB es LL#####
        case "carnet":
            re = /^[A-Z]{2}[0-9]{6}$/;
            if(re.test(dato)){
                valido = true;
            }
            break;
        //El formato de número telefónico en El Salvador es ####-####
        case "telefono":
            re = /^\\d{4}-\\d{4}$/;
            if(re.test(dato)){
                valido = true;
            }
            break;
        default:
            alert("El tipo de dato seleccionado no puede ser procesado");
            break;
    }
    if(valido == true){
```

```

    alert("El valor " + dato + " de " + tipo + " ingresado es válido");
  }
  else if(valido == false){
    alert("El valor " + dato + " de " + tipo + " ingresado es inválido");
  }
}
//Asociando función que manejará el evento load al cargar la página
if(window.addEventListener){
  window.addEventListener("load", iniciar, false);
}
else if(window.attachEvent){
  window.attachEvent("onload", iniciar);
}

```

Resultado:

Ejemplo #2: El siguiente ejemplo permite ingresar un deporte en una caja de texto y mientras ingresa el nombre del deporte, si está presente en el cuadro de lista, comienza a mostrar las coincidencias filtrando en el momento en el cuadro de lista. Adicionalmente se puede hacer doble clic en el cuadro de lista cuando todavía no se ha terminado de escribir el nombre del deporte e inmediatamente llena el cuadro de texto donde se estaba ingresando el nombre del deporte. Para esto se utilizan manejadores de evento en la caja de texto con keyup y dblclick en el cuadro de lista.

Guión 1: busqueda.html

```

<!DOCTYPE html>
<html lang="es">
<head>
  <title>Deportes olímpicos</title>
  <meta charset="utf-8" />
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.0/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-
gH2ylJqKdNHPEq0n4MqA/HGKIhSkiHel5AyhkYV8i59U5AR6csBvApHHNI/vl1Bx" crossorigin="anonymous">
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.2.0/dist/js/bootstrap.bundle.min.js" integrity="sha384-
A3rJD856KowSb7dwlZdYEkO39Gagi7vlsF0jrRAoQmDKKtQBHUuLZ9AsSv4jD4Xa" crossorigin="anonymous"></script>
  <script src="js/busqueda.js"></script>
</head>
<body>
  <div class="jumbotron jumbotron-fluid bg-success text-white">
    <div class="container">
      <h1 class="display-4">Deportes</h1>
      <p class="lead">¿Qué deportes olímpico te gusta más?</p>
    </div>
  </div>
  <div class="container">
    <form id="frmsearchsport" name="frmsearchsport">

    <fieldset id="box-form">
      <div class="form-group mb-2">
        <label class="sr-only" for="txtsport">Ingresa un deporte:</label>
        <input class="form-control" type="text" id="txtsport" name="busca" size="24"><br/>
      </div>
      <div class="form-group mb-2"></div>
      <label class="sr-only" for="lstsports">Listado:</label>
      <select class="form-control" name="lstsports" id="lstsports" size="24">
        </select>
    </fieldset>
  </div>

```



```
</div>
</fieldset>
</form>
</div>
</body>
</html>
```

Crear carpeta js y dentro de ella cree y coloque el siguiente script.

Guión 2: busqueda.js

```
//Definir la matriz o arreglo con los datos
var lista = new Array('Aerobic', 'Ajedrez', 'Atletismo', 'Balonmano', 'Béisbol', 'Badminton',
    'Baloncesto', 'Boxeo', 'Ciclismo', 'Equitación', 'Esgrima', 'Footing',
    'Fútbol', 'Gimnasia artística', 'Gimnasia rítmica', 'Judo', 'Lanzamiento de disco',
    'Lanzamiento de jabalina', 'Levantamiento de pesas', 'Lucha', 'Natación',
    'Natación sincronizada', 'Patinaje', 'Pentatlón', 'Remo', 'Salto alto',
    'Salto largo', 'Tenis', 'Tenis de mesa', 'Tiro con arco', 'Voleibol', 'Waterpolo'
);

//Función iniciar desde donde se controlarán los manejadores de evento a utilizar
function iniciar(){
    //Creando el controlador de evento para el evento doble clic sobre los elementos del cuadro de lista
    var lista = document.getElementById("Istsports");
    //Cargando los datos en el cuadro de lista al cargar la página
    cargarLista();
    if(lista.addEventListener){
        lista.addEventListener("dblclick", function(){
            cargarTexto(this.options[this.selectedIndex].text);
        }, false);
    }
    else if(lista.attachEvent){
        lista.attachEvent("ondblclick", function(){
            cargarTexto(this.options[this.selectedIndex].text);
        });
    }
    //Capturando el campo de texto sobre el que se ingresan los deportes mediante pulsación de tecla
    var texto = document.getElementById("txtsport");
    if(texto.addEventListener){
        texto.addEventListener("keyup", buscar, false);
    }
    else if(texto.attachEvent){
        texto.attachEvent("onkeyup", buscar);
    }
}

function cargarLista() {
    // Cargamos el combo
    for(x=0;x<lista.length;x++)
        document.frmsearchsport.Istsports[x] = new Option(lista[x]);
}

function cargarTexto(elemento){
    if(elemento!=null && elemento!=undefined && elemento!="")
        document.frmsearchsport.txtsport.value = elemento;
}

function buscar() {
    limpiarLista();
    // Se obtiene el valor del texto
    var str = document.frmsearchsport.txtsport.value;

    // Crear la expresión regular
    var expr = new RegExp("^" + str, "i");
    // Recorrer la lista. Si la expresión regular es OK
    y = 0;
```

Resultado:

Ejemplo #3: El siguiente ejemplo permite encontrar dentro de un documento todos los elementos que existen que son de una clase seleccionada mediante un campo select de formulario. Cuando ha ingresado en un elemento de tipo texto el nombre del elemento en el que desea buscar o indicando que buscará en todos los elementos por la clase. Se utiliza una expresión regular, en este caso con el constructor `new RegExp()` por el hecho de que el patrón de expresión regular se determina hasta que el usuario da clic en el botón encontrar del formulario; es decir, la expresión regular no es fija.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title></title>
  <meta charset="utf-8" />
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.0/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-gH2ylJqKdNHPEq0n4Mqa/HGKIhSkIHeL5AyhkYV8i59U5AR6csBvAPHHNI/vl1Bx" crossorigin="anonymous">
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.2.0/dist/js/bootstrap.bundle.min.js" integrity="sha384-A3rJD856KowSb77dwldZdYekO39Gagi7vlsF0jrRAoQmDCKtQBHUuLZ9AsSv4jD4Xa" crossorigin="anonymous"></script>
  <!--libreria para iconos-->
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.9.1/font/bootstrap-icons.css">
  <script src="js/findclass.js"></script>
</head>
<body>
<section>
<article>
<div class="samurai ninja">
</div>
<div class="ninja samurai">
</div>
<div></div>
<span class="samurai ninja viking"></span><br />
<div class="container">
<section class="tabs blue">
```

```

<h2 for="tab1">Recuento de clases</h2>
<form name="frmclass" id="classname">
  <label class="form-label">Qué clase desea buscar en el documento.</label>
  <div class="input-group mb-4 mr-sm-2">
    <div class="input-group-prepend">
      <div class="input-group-text bg-success text-white">
        <i class="bi bi-filetype-html"></i>
      </div>
    </div>
    <input class="form-control" type="text" name="txtname" id="txtname" list="tags" placeholder="(Ingrese una clase)"
required class="field" />
    <datalist id="tags">
      <option value="ninja" class="field"></option>
      <option value="samurai" class="field"></option>
      <option value="viking" class="field"></option>
    </datalist>
  </div>

  <div class="input-group mb-4 mr-sm-6">
    <div class="input-group-prepend3">
      <div class="input-group-text bg-warning text-white"><i class="bi bi-filetype-css"></i></div>
    </div>
    <select class="form-control" name="selelement" id="selelement" required>
      <option disabled="" selected="">(Seleccione una etiqueta)</option>
      <option value="*">TODO</option>
      <option value="div">DIV</option>
      <option value="span">SPAN</option>
    </select>
  </div>
  <input class="btn btn-primary mb-3 type="button" name="btnSend" id="btnSend" value="Encontrar" />
</form>
</section>
</div>
</article>
</section>
</body>
</html>

```

Crear carpeta js y dentro de ella cree y coloque el siguiente script.

Guión 2: findclass.js

```

function iniciar(){
  var classname, tag="";
  var btnfind = document.getElementById("btnSend");
  if(btnfind.addEventListener){
    btnfind.addEventListener("click", function(){
      //Es de suponer que el valor del campo de texto y del campo select del formulario
      //estará establecido cuando se haga clic sobre el botón Encontrar
      classname = document.frmclass.txtname.value;
      tag = document.frmclass.selelement[document.frmclass.selelement.selectedIndex].value;
      alert("Se han encontrado " + findClassInElements(classname, tag) + " elementos " + tag.toString() + " con la clase " +
classname);
    }, false);
  }
  else if(btnfind.attachEvent){
    btnfind.attachEvent("onclick", function(){
      //Es de suponer que el valor del campo de texto y del campo select del formulario
      //estará establecido cuando se haga clic sobre el botón Encontrar
      classname = document.frmclass.txtname.value;
      tag = document.frmclass.selelement[document.frmclass.selelement.selectedIndex].value;
      alert("Se han encontrado " + findClassInElements(classname, tag) + " elementos " + tag.toString() + " con la clase " +
classname);
    });
  }
}

function findClassInElements(className, type){

```

```

//Detectar el elemento sobre el que se realizará la búsqueda de la clase.
//Puede ser un elemento en particular o todos los elementos
var elems = document.getElementsByTagName(type || "*");
//En este caso el patrón de la expresión regular incluirá una variable,
//por lo tanto estará obligado a usar el constructor RegExp.
var regex = new RegExp("(^|\\s)" + className + "(\\s|$)");
var results = new Array();
//El ciclo recorre toda la página comprobando en el elemento enviado a la función
//si la clase está presente o no en dicho elemento. Si lo está se guarda dentro
//de la matriz o arreglo results.
for(var i=0, length=elems.length; i<length; i++){
    if(regex.test(elems[i].className)){
        results.push(elems[i]);
    }
}
//Se devuelve el total encontrado
return results.length;
}

//Asociando función que manejará el evento load al cargar la página
if(window.addEventListener){
    window.addEventListener("load", iniciar, false);
}
else if(window.attachEvent){
    window.attachEvent("onload", iniciar);
}

```

Resultado:

Recuento de clases

Qué clase desea buscar en el documento.

 (Ingrese una clase)

 TODOS

Encontrar

V Discusión de resultados.

1. Realice un validador de expresiones regulares para los siguientes casos: a) direcciones IPv4 (tome en cuenta que las direcciones IP llegan como máximo hasta 255.255.255.255), b) direcciones web (URLs) tomando en cuenta protocolos como http, https y ftp y que las direcciones como `//html5shiv.googlecode.com/svn/trunk/html5.js` también son válidas, y c) direcciones de correo electrónico. La implementación debe ser exactamente igual que el problema número 1 en el que se selecciona el tipo de validación que queremos y un campo de texto a la par donde se ingresa esa dirección. Haga que aparezca el mensaje de alerta apropiado si la URL o la dirección IP es válida o no.
2. Realizar una aplicación web que permita validar el tipo de archivo que se intenta subir a un servidor web, mediante el uso de un campo de formulario de tipo file. El control se simula usando un elemento label y ocultando el verdadero campo input type="file". La validación que se hace usando una expresión regular que verifica el nombre del archivo y la extensión, para que sólo acepte archivos de imagen que se puedan publicar en la web, como .jpg, .png y .gif.
3. Siguiendo el ejemplo del ejercicio 2, cree un script que le permita buscar palabras de un diccionario para mostrar su definición en un área de texto. La definición deberá cargarse en el área de texto al hacer doble clic sobre la palabra deseada en el cuadro de lista o al hacer clic en un botón denominado mostrar definición. Como el cuadro de lista contendrá muchas palabras deberá utilizar un cuadro de texto para filtrar las palabras a medida se vaya escribiendo en el cuadro de texto. Puede utilizar una interfaz como la que se muestra a continuación. Como mínimo deberá presentar unas 80 palabras con sus definiciones. Utilice palabras con la mayor parte de letras del alfabeto español.

Palabra a buscar:

Palabras coincidentes:

- Caballo
- Cabaña
- Cabina
- Cadáver
- Cadena
- Cadmio
- Cuarto
- Cuarzo
- Culpa
- Cultivo
- Cultura

Definición:

* adj. Que dice relación a una cualidad o accidente.
*Gram. Nombre que califica al sustantivo o delimita su extensión.

Mostrar definición

VI Bibliografía.

- Powell, Thomas / Schneider, Fritz. JavaScript Manual de Referencia. 1ra Edición. Editorial McGraw-Hill. 2002. Madrid, España.
- Flanagan, David. JavaScript La Guía Definitiva. 1ra Edición. Editorial ANAYA Multimedia. 2007. Madrid, España.
- McFedries, Paul. JavaScript Edición Especial. 1ra Edición. Editorial Prentice Hall. 2002. Madrid, España.
López Quijado, José. Domine JavaScript. 2da Edición. Editorial Alfaomega RAMA. Mayo 2007. México D.F.
http://developer.mozilla.org/es/docs/Gu%C3%ADa_JavaScript_1.5:Expresiones_Regulares.